

1.Program for congestion control using the Leaky Bucket algorithm

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <unistd.h>

using namespace std;

#define NO_OF_PACKETS 10

int arc4rand(int a)
{
    int m = (rand() % 10) % a;
    return (m == 0) ? 1 : m;
}

int main()
{
    int packet_sz[NO_OF_PACKETS];
    int i, clk, b_size, o_rate, p_sz_rm = 0, p_time, op;

    srand(time(0));

    for (i = 0; i < NO_OF_PACKETS; i++)
    {
        packet_sz[i] = arc4rand(6) * 10;
        cout << "Packet[" << i << "] = "
              << packet_sz[i] << " bytes" << endl;
    }

    cout << "\nEnter the Output rate: ";
    cin >> o_rate;

    cout << "Enter the Bucket Size: ";
    cin >> b_size;

    for (i = 0; i < NO_OF_PACKETS; i++)
    {
        // Check if packet size exceeds bucket capacity
```

```

if (packet_sz[i] > b_size)
{
    cout << "\nIncoming packet size ("
        << packet_sz[i]
        << ") is Greater than bucket capacity - "
        << "PACKET REJECTED!" << endl;
}
else
{
    p_sz_rm += packet_sz[i];

    cout << "\nIncoming packet size: "
        << packet_sz[i] << endl;

    cout << "Bytes remaining to transmit: "
        << p_sz_rm << endl;

    // Generate transmission time
    p_time = arc4rand(4) * 10;

    cout << "Time left for transmission: "
        << p_time << " units" << endl;

    // Transmission
    for (clk = 10; clk <= p_time; clk += 10)
    {
        sleep(1);

        if (p_sz_rm)
        {
            // Compare remaining packet size
            // with output rate
            if (p_sz_rm <= o_rate)
            {
                op = p_sz_rm;
                p_sz_rm = 0;
            }
            else
            {
                op = o_rate;
                p_sz_rm -= o_rate;
            }

            cout << "Packet of size "

```

```

        << op
        << " Transmitted" << endl;

        cout << "Bytes Remaining to Transmit: "
              << p_sz_rm << endl;
    }
    else
    {
        cout << "Time left for transmission: "
              << p_time - clk
              << " units" << endl;

        cout << "No packets to transmit!" << endl;
    }
}
}
}
}

return 0;
}

```



"C:\Users\admin\Desktop\165 X



```
Packet[0] = 30 bytes
Packet[1] = 10 bytes
Packet[2] = 20 bytes
Packet[3] = 10 bytes
Packet[4] = 30 bytes
Packet[5] = 10 bytes
Packet[6] = 10 bytes
Packet[7] = 50 bytes
Packet[8] = 10 bytes
Packet[9] = 20 bytes
```

```
Enter the Output rate: 10
Enter the Bucket Size: 30
```

```
Incoming packet size: 30
Bytes remaining to transmit: 30
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 20
```

```
Incoming packet size: 10
Bytes remaining to transmit: 30
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 20
```

```
Incoming packet size: 20
Bytes remaining to transmit: 40
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 30
```

```
Incoming packet size: 10
Bytes remaining to transmit: 40
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 30
```

```
Incoming packet size: 30
Bytes remaining to transmit: 60
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 50
```

```
Incoming packet size: 10
Bytes remaining to transmit: 60
Time left for transmission: 20 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 50
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 40
```

```
Incoming packet size: 10
Bytes remaining to transmit: 50
Time left for transmission: 30 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 40
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 30
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 20
```

```
Incoming packet size (50) is Greater than bucket capacity - PACKET REJECTED!
```

```
Incoming packet size: 10
Bytes remaining to transmit: 30
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 20
```

```
Incoming packet size: 20
Bytes remaining to transmit: 40
Time left for transmission: 10 units
Packet of size 10 Transmitted
Bytes Remaining to Transmit: 30
```

2.TCP client-server file transfer program.

```
from socket import *

serverName = "127.0.0.1"
serverPort = 12000

serverSocket = socket(AF_INET, SOCK_STREAM)

serverSocket.bind((serverName, serverPort))

serverSocket.listen(1)

print("The server is ready to receive")

while True:

    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()

    file = open(sentence, "r")

    l = file.read(1024)

    connectionSocket.send(l.encode())

    file.close()

    connectionSocket.close()
```

```
from socket import *

serverName = "127.0.0.1"
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_STREAM)

clientSocket.connect((serverName, serverPort))
```

```
sentence = input("Enter file name: ")

clientSocket.send(sentence.encode())

filecontents = clientSocket.recv(1024).decode()

print("From Server:", filecontents)

clientSocket.close()
```

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin\Desktop\165\cn>python server.py
The server is ready to receive
█
```

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin\Desktop\165\cn>python client.py
Enter file name: tests.txt
From Server: Hello!
This is my test file.
This file is being sent using TCP sockets.

C:\Users\admin\Desktop\165\cn>█
```

3.UDP client-server file transfer program.

```
from socket import *

serverPort = 12000

serverSocket = socket(AF_INET, SOCK_DGRAM)

serverSocket.bind(("127.0.0.1", serverPort))

print("The server is ready to receive")

while True:

    sentence, clientAddress = serverSocket.recvfrom(2048)

    fileName = sentence.decode("utf-8")

    try:

        file = open(fileName, "r")

        fileContents = file.read(2048)

        serverSocket.sendto(
            fileContents.encode("utf-8"),
            clientAddress
        )

        print("Sent back to client")

        file.close()

    except FileNotFoundError:

        message = "File not found"

        serverSocket.sendto(
            message.encode("utf-8"),
            clientAddress
```

```

    )

from socket import *

serverName = "127.0.0.1"
serverPort = 12000

clientSocket = socket(AF_INET, SOCK_DGRAM)

sentence = input("Enter file name: ")

clientSocket.sendto(
    sentence.encode("utf-8"),
    (serverName, serverPort)
)

filecontents, serverAddress = clientSocket.recvfrom(2048)

print("From Server:", filecontents.decode("utf-8"))

clientSocket.close()

```

```

PS C:\Users\admin\Desktop\165\cn lab udp> python ServerUDP.py
The server is ready to receive
Sent back to client

```

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\admin\Desktop\165\cn lab udp> python ClientUDP.py
Enter file name: tests.txt
From Server: Hello!
This is a UDP file transfer program.
PS C:\Users\admin\Desktop\165\cn lab udp>

```